UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO** **FACULTAD DE INGENIERÍA** **ESCUELA DE COMPUTACION**
CICLO 02	**GUIA DE LABORATORIO Nº9** **Nombre de la practica**: Procedimientos Almacenados **Lugar de ejecución**: Laboratorio de Informática **Tiempo estimado**: 2 horas **Materia**: Análisis y diseño de sistemas y Base de Datos

I. Objetivos

Qué el estudiante:

1. Crear, probar y desplegar un procedimiento almacenado
2. Implementar la programación con comandos SQL

II. Introducción Teórica

PROCEDIMIENTOS ALMACENADOS

Transact-SQL no es realmente un lenguaje de programación similar a las herramientas de tercera y cuarta generación sin embargo permite utilizar SQL para realizar tareas complejas que requieren saltos, bucles, decisiones. **Transact-SQL** se utiliza a menudo en la creación de procedimientos almacenados y triggers (desencadenadores) de tal forma que las aplicaciones clientes que se conectan a SQL Server solo se preocupan por la presentación de los datos para el usuario final, mientras que la lógica de los procesos se maneja en el servidor.

El control de flujo en Transact-SQL

INSTRUCCIÓN	DESCRIPCIÓN
IF...ELSE	Define una decisión.
GOTO etiqueta	Define un salto incondicional
WAITFOR	Establece un tiempo para la ejecución de una instrucción. El tiempo puede ser un intervalo de retardo o un instante especificado de ejecución (una hora concreta del día)
WHILE	Bucle básico de SQL
BREAK	Acompaña al bucle WHILE y le indica finalizarlo inmediatamente.
CONTINUE	Acompaña al bucle WHILE y le indica continuar con la siguiente iteración.
RETURN [n]	Salida incondicional del procedimiento o proceso por lotes, se puede definir un número entero como estado devuelto y puede asignarse a cualquier variable.
BEGIN...END	Utilizado en conjunto con IF..ELSE o WHILE para agrupar un conjunto de instrucciones.
CASE	Implementada en la instrucción SELECT y UPDATE y permite realizar consultas y actualizaciones condicionales.
PRINT	Es una instrucción que se utiliza para imprimir un dato en la pantalla, la sintaxis es: **PRINT** "cadena"; cadena puede ser también una variable de tipo varchar. Por ejemplo: **PRINT** 'Hola a todos'

Los procedimientos almacenados son conjuntos de sentencias en leguaje Transact SQL que pueden almacenarse en el propio servidor. Los procedimientos almacenados de SQL Server, son más potentes, porque permiten almacenar funciones y procedimientos compuestos por varias instrucciones, introducir saltos, bucles, etc.

Creación de procedimientos almacenados (Store Procedures)

La instrucción general para crear procedimientos almacenados es la siguiente:

CREATE PROC|PROCEDURE nombre_proc parametros
AS
INSTRUCCION SQL

Es necesario aclarar, que un procedimiento almacenado puede recibir parámetros de entrada y devolver parámetros de salida.

Por ejemplo, podemos crear un procedimiento para recuperar el nombre de un cliente (), cuyo código se pasa por parámetro.

```
CREATE  PROCEDURE  ObtenerNombre
      @cliente_id  nchar(5)
AS
      SELECT      ContactName
      FROM        Customers
      WHERE       CustomerID = @cliente_id
GO
```

Con esta sentencia, se crea un procedimiento almacenado, de nombre ObtenerNombre, al que se le pasa un parámetro, llamado @cliente_id, de tipo nchar tamaño 5, que realiza una consulta para obtener el nombre del cliente de la tabla Customers, cuyo código (CustomerID) coincida con el parámetro. De esta forma, si queremos obtener el nombre del Cliente cuyo código sea 'ALFKI', deberemos ejecutar el procedimiento pasándole como argumento este valor.

Las llamadas a procedimientos almacenados se pueden realizar de las siguientes formas:

- Pasando los argumentos en el mismo orden que en el que se han declarado.

```
EXEC  ObtenerNombre  'ALFKI'
```

- Pasando los argumentos nombrados. En este caso no hace falta que los parámetros vayan en el mismo orden.

```
EXEC  ObtenerNombre  @cliente_id='ALFKI'
```

Parámetros por referencia

Si ejecutamos las anteriores sentencias, obtendremos el resultado directamente en la ventana que tengamos abierta en SQL Server. Pero ¿qué pasa si queremos obtener un parámetro de salida, como resultado de la ejecución del procedimiento? La solución para este caso es utilizar la palabra reservada OUTPUT para los argumentos de salida.

Si, por ejemplo, queremos obtener el número de títulos que tenemos en la base de datos, crearemos el siguiente procedimiento almacenado:

```
CREATE  PROCEDURE  num_productos
      @productos  int  OUTPUT
AS
      SELECT  *  FROM  Products
      SELECT  @productos  =  @@ROWCOUNT
      RETURN (0)
```

Vamos a estudiar el anterior ejemplo:
Detrás de la palabra reservada PROCEDURE damos el nombre del procedimiento almacenado, y a continuación proporcionamos los parámetros, junto con su tipo (que en este caso es entero), y diremos si éstos son de salida, en cuyo caso especificamos la palabra reservada **OUTPUT** a continuación. Tras la palabra reservada **AS** se codifica el cuerpo del procedimiento.

Primero contamos todas las filas de la tabla Products, realizando un SELECT * FROM Products.
A continuación, devolvemos en el parámetro @productos el valor obtenido, utilizando *@@ROWCOUNT.*

Para ejecutar este procedimiento ejecutamos las siguientes sentencias (las tres al mismo tiempo):

```
DECLARE  @productos  int

EXEC  num_productos  @productos  OUTPUT

SELECT  [Total  de  productos]  =  @productos
```

En el resultado de la consulta nos mostrará los registros de la tabla **Products** y en una nueva fila el valor de la variable @productos en una columna llamada Total de productos.

	ProductID	ProductName	SupplierID	CategoryID	QuantityPerUnit	UnitPrice	UnitsInStock
1	1	Chai	1	1	10 boxes x 20 bags	18.00	39
2	2	Chang	1	1	24 - 12 oz bottles	19.00	17
3	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10.00	13
4	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22.00	53

	Total de productos
1	77

Borrar Procedimientos Almacenados

Si queremos borrar un procedimiento almacenado, ejecutaremos la sentencia DROP PROCEDURE, seguido del nombre del procedimiento. Por ejemplo, si queremos borrar el procedimiento almacenado, creado en el anterior ejemplo, escribiremos el Código:

```
DROP  PROCEDURE  num_productos
```

Control de errores usando bloques Try – Catch

Un bloque TRY – CATCH detecta los errores de ejecución que tienen una gravedad mayor a 10 y que no cierran la conexión a la base de datos utilizada. Cabe mencionar que ésta estructura de control puede ser tanto implementada en procedimientos almacenados, consultas y funciones.

La implementación de control de error en T-SQL es similar a la implementación en lenguajes de programación como C#.

Para poder regresar la información del error tenemos nuevas funciones en SQL Server, veamos cuales son:

- **ERROR_NUMBER():** Devuelve el número de error.
- **ERROR_MESSAGE():** Devuelve el texto completo del mensaje de error. El texto incluye los valores suministrados para los parámetros sustituibles, como longitudes, nombres de objeto u horas.
- **ERROR SEVERITY**(): Devuelve la gravedad del error.
- **ERROR_STATE():** Devuelve el número de estado de error.
- **ERROR_LINE():** Devuelve el número de línea donde se produjo el error.
- **ERROR_PROCEDURE():** Devuelve el nombre del procedimiento donde se produjo el error.

Estructura de un TRY – CATCH

Un bloque TRY-CATCH comienza con la sentencia BEGIN TRY, posteriormente todas las sentencias criticas donde se necesite control algún error de ejecución, finalizando con la sentencia END TRY.

Inmediatamente después de cerrar el bloque TRY se debe escribir la sentencia BEGIN CATCH, esta sección permite manejar el error generado en la sección critica. Las sentencias de manejo de errores quedan totalmente a discreción de la lógica implementada de la solución. Finalmente se cierra el bloque CATCH con la sentencia END CATCH.

```
BEGIN TRY

       --Instrucciones

END TRY
BEGIN CATCH

    --Captura de errores

END CATCH
```

Ejemplo de uso TRY – CATCH

Para este ejemplo, se provocará un error intencionalmente con la finalidad de visualizar un mensaje el cual reporte el error que ha ocurrido **(Esto es un bloque anónimo que puede ejecutarse en la pestaña de ejecución de SQL).**

```
BEGIN TRY
       DECLARE @valor INT;
       SET @valor = 7;
       SELECT @valor/0
END TRY
BEGIN CATCH
       SELECT 'No se puede dividir el valor entre CERO'
END CATCH
```

En este ejemplo se esta provocando un error aritmético explícitamente dividiendo un valor entre CERO. El error es capturado en el bloque CATCH y ejecuta la lógica indicada para el manejo del error, en este caso, un mensaje el cual especifica que no se puede realizar la operación.

	(No column name)

	(No column name)
1	No se puede dividir el valor entre CERO

Ahora se especificará en el bloque catch que se requiere retornar más información sobre el error ocurrido, para ello el siguiente ejemplo:

```
BEGIN TRY
	DECLARE @valor INT;
	SET @valor = 7;
	SELECT @valor/0
END TRY
BEGIN CATCH
SELECT
	ERROR_NUMBER() AS Numero_de_Error,
	ERROR_SEVERITY() AS Gravedad_del_Error,
	ERROR_STATE() AS Estado_del_Error,
	ERROR_PROCEDURE() AS Procedimiento_del_Error,
	ERROR_LINE() AS Linea_de_Error,
	ERROR_MESSAGE() AS Mensaje_de_Error;
END CATCH
```

De esta forma el bloque CATCH mostrara el numero de error, la gravedad y principalmente una idea de lo ocurrido en el mensaje de error.

	(No column name)					
	Numero_de_Error	Gravedad_del_Error	Estado_del_Error	Procedimiento_del_Error	Linea_de_Error	Mensaje_de_Error
1	8134	16	1	NULL	4	Divide by zero error encountered.

III. Requerimientos

Nº	Cantidad	Descripción
1	1	Guía de Laboratorio #9 de Base de Datos
2	1	Maquina con SQL Server

IV. Procedimiento

Parte 1: Iniciando sesión desde SQL Server Managment Studio

1. Hacer clic en el botón **Inicio**

2. Hacer clic en la opción **Todos los programas** y hacer clic en **Microsoft SQL Server**

Para conectarse con el servidor de base de datos elija los siguientes parámetros de autenticación:

- **Tipo de servidor:** Database Engine
- **Nombre del servidor:** Colocar el nombre del servidor local, por ejemplo, PC**NumMaquina-SALA#** Nota: NumMaquina es el número de la maquina local
- **Autenticación**: SQL Server Authentication
- **Login:** sa
- **Password:** 123456

3. Luego, presione Ctrl+N o de clic en **"New Query"** en la barra de trabajo estándar.

Parte 2: Ejemplos de Procedimientos almacenados

Ejercicio 1

1. Utilice la base de datos **NorthWind**

2. Vamos a crear un procedimiento almacenado que ingrese un nuevo registro a la tabla **Categories** solo si el Código o el Nombre de la categoría que se le envían como parámetros no existan en la tabla.

```
CREATE PROC sp_Insertar_CategoriasSuCarnet
      @ID INT,
      @NombreCategoria VARCHAR(15)
AS
IF( SELECT COUNT(*) FROM Categories
      WHERE CategoryID = @ID OR CategoryName = @NombreCategoria) = 0
BEGIN
      INSERT INTO Categories(CategoryName)
      VALUES(@NombreCategoria)
END
ELSE
PRINT 'Error, la categoría ya existe'
GO
```

3. Ejecutemos el procedimiento anterior comprobando si funciona correctamente, ejecutar cada instrucción individualmente:

```
--Ejercutar primer
EXEC sp_Insertar_CategoriasSuCarnet 1, 'Alimentos'
--Segundo
EXEC sp_Insertar_CategoriasSuCarnet 9, 'Alimentos'
--Tercero
EXEC sp_Insertar_CategoriasSuCarnet 10, 'Alimentos'
```

Nota: si los datos del ejemplo anterior existen en la tabla probar con otros datos

4. Hacer un SELECT y verifique cuantos registros se agregaron a la tabla **Categories**

```
SELECT * FROM Categories
```

	CategoryID	CategoryName	Description	Picture
1	1	Beverages	Soft drinks, coffees, teas, beers, and ales	0x151C2F00020000
2	2	Condiments	Sweet and savory sauces, relishes, spreads, and ...	0x151C2F00020000
3	3	Confections	Desserts, candies, and sweet breads	0x151C2F00020000
4	4	Dairy Products	Cheeses	0x151C2F00020000
5	5	Grains/Cereals	Breads, crackers, pasta, and cereal	0x151C2F00020000
6	6	Meat/Poultry	Prepared meats	0x151C2F00020000
7	7	Produce	Dried fruit and bean curd	0x151C2F00020000
8	8	Seafood	Seaweed and fish	0x151C2F00020000
9	9	Alimentos	NULL	NULL

Ejercicio 2

1. En el siguiente procedimiento hacemos uso de la sentencia CASE para verificar si existen clientes en una determinada ciudad la cual su dato se pasa como parámetro.

```
CREATE PROCEDURE sp_HayClientesSuCarnet
     @ciudad VARCHAR(15)
AS
SELECT
CASE (SELECT COUNT(*) FROM Customers WHERE City = @ciudad)
WHEN 0 THEN 'No hay clientes en la ciudad de ' + @ciudad
ELSE 'Hay clientes en la ciudad de ' + @ciudad
END
```

2. Ejecutemos el procedimiento anterior comprobando si funciona correctamente:

```
EXEC sp_HayClientesSuCarnet 'Barcelona'
EXEC sp_HayClientesSuCarnet @ciudad = 'New York'
```

Ejercicio 3.

1. En el siguiente procedimiento se calcula el total de ventas de los pedidos que están almacenados en la tabla **Order Details**

```
CREATE PROC sp_Total_Ventas_SuCarnet
 AS
SELECT
     OrderID,
     SUM(CONVERT( MONEY, (UnitPrice*Quantity*(1-Discount)/100))*100) AS Total
FROM [Order Details]
GROUP BY OrderID
GO
```

La función CONVERT () es una función general que convierte una expresión de un tipo de dato a otro.

2. Ejecutar el procedimiento

```
EXEC sp_Total_Ventas_SuCarnet
```

3. Ahora vamos a alterar o modificar el procedimiento para que reciba como parámetro de entrada una orden o pedido especifico

```
ALTER PROC sp_Total_Ventas_SuCarnet
     @Cod_orden INT
AS
SELECT
     OrderID,
     SUM(CONVERT( MONEY, (UnitPrice*Quantity*(1-Discount)/100))*100) AS Total
FROM [Order Details]
WHERE OrderID = @Cod_orden
GROUP BY OrderID
GO
```

4. Ejecutar el procedimiento

```
EXEC sp_Total_Ventas_SuCarnet 10248
```

5. Ahora ejecutamos el mismo procedimiento con otro código de pedido

```
EXEC sp_Total_Ventas_SuCarnet 10242
```

Analice y comente la diferencia entre los resultados obtenidos, como se dará cuenta el OrderID 10242 no existe en la tabla, pero al ejecutarse se muestran las columnas con datos vacíos

6. Se puede modificar el procedimiento de la siguiente manera, en donde se muestra un mensaje indicando que no existe esa orden

```
ALTER PROC sp_Total_Ventas_SuCarnet
      @Cod_orden INT
AS
DECLARE @total INT -- Declaración de variables locales
SELECT @total = COUNT(*) FROM [Order Details] WHERE OrderID = @Cod_orden
IF(@total >= 1) --Evaluar condición
BEGIN
      SELECT
            OrderID,
            SUM(CONVERT( MONEY, (UnitPrice*Quantity*(1-Discount)/100))*100) AS Total
      FROM [Order Details]
      WHERE OrderID = @Cod_orden
      GROUP BY OrderID
END
ELSE
BEGIN
      PRINT 'La orden no existe el código es: ' + CONVERT(VARCHAR(10), @Cod_Orden)
END
GO
```

7. Ejecutar el procedimiento

```
--Código de pedido que si existe en la tabla Order Details
EXEC sp_Total_Ventas_SuCarnet 10248
--Código de pedido que no existe en la tabla Order Details
EXEC sp_Total_Ventas_SuCarnet 10242
```

8. Verifique y analice los resultados

9. Guardar el script con el nombre **Guia9_BasedeDatosSucarnet.sql**

V. Ejercicio complementario

Creación de Vistas

Ejercicio 1

La base de datos a utilizar es: **Northwind**

Crear las siguientes **vistas**:

a. Mostrar el código del producto, el nombre del producto y el precio por unidad de todos los productos de la empresa.

b. Mostrar todos los productos cuya categoría sea Beverages.

c. Mostrar los datos del cliente y las fechas de las ordenes que estos han realizado

d. Cuantos productos existen por cada categoría

e. Mostrar el promedio de los precios unitarios de las categorías: Produce y Confections

Ejercicio 2

Crear un procedimiento almacenado que calcule y muestre la edad de un empleado (Tabla Employees, campo BirthDate), puede utilizar la siguiente instrucción para calcular la edad:

(CAST(DATEDIFF(dd, BirthDate,GETDATE())/365.25 as int))

Y por medio de una sentencia CASE deberá calcular y mostrar si el empleado esta por retirarse (mayores de 60 y menores o igual a 70 años), si ya está jubilado (mayores a 70 años) o le falta otros años para trabajar (menores o iguales a 60 años)

El procedimiento recibe como parámetros el código del empleado (EmployeeID) si el empleado no existe deberá mostrar un mensaje indicándolo.

VI. Análisis de resultados

Uso de Procedimiento almacenado

```
CREATE DATABASE Bodega
GO

USE Bodega
GO

CREATE TABLE PRODUCTO
(
idprod CHAR(7) PRIMARY KEY,
descripcion VARCHAR(25),
existencias INT, --cantidad de producto existentes
precio DECIMAL(10,2) NOT NULL, --precio costo
preciov DECIMAL(10,2) NOT NULL, --precio venta
ganancia AS preciov - precio, --campo para calcular ganancia
CHECK(preciov>precio) --precio venta tiene que ser mayor al precio de compra
)
GO

CREATE TABLE PEDIDO
(
idpedido CHAR(7),
idprod CHAR(7),
cantidad INT -- cantidad de unidades vendidas del producto en el pedido
FOREIGN KEY(idprod) REFERENCES PRODUCTO(idprod)
)
```

La tabla PRODUCTO contiene información general de los productos y la tabla PEDIDO contiene la información del pedido que se realiza de un cierto producto, llenar las tablas con información.

Ejercicios:

1. Crear un procedimiento almacenado que ingrese los valores en la tabla **PRODUCTOS**, y deberá verificar que el código y nombre del producto no exista para poder insertarlo, en caso de que el código o el nombre del producto ya exista enviar un mensaje que diga “ESTE PRODUCTO YA HA SIDO INGRESADO”.

2. Crear un procedimiento almacenado que permita realizar un pedido EN LA TABLA PEDIDO, este procedimiento deberá verificar si el código del producto ingresado existe en la tabla PRODUCTO en caso de que no se encuentre deberá mostrar un mensaje, así como se muestra a continuación "ESTE PRODUCTO NO EXISTE ", además si la cantidad a pedir del producto es mayor a la existencia del producto deberá mostrar un mensaje que diga "EXISTENCIA DEL PRODUCTO INSUFICIENTE".
   En caso de que la cantidad a pedir sea menor o igual deberá modificar (o actualizar) el valor de la existencia del producto.

VI. Referencia Bibliográfica

1. La Biblia de SQL Server 2005
   Madrid, España: Anaya, 2006
   Autor: Mike Gundelerloy y Joseph L. Jorden
   Biblioteca UDB – Clasificación: 005.361 G975 2006

2. Fundamentos: de Programación con SQL Server 2005
   Madrid, España: ANAYA, 2006
   Autor: Roberto Vieira
   Biblioteca UDB – Clasificación: 005.361 V665

3. SQL Server 2008
   Madrid, España: ANAYA, 2009
   Autor: Francisco Charte Ojeda
   Biblioteca UDB – Clasificación: 005.361 Ch436 2009